Rajaprabhu A

Eye gaze controlled wheel chair using machinelearning and Deeplearning

Project Report

Computer Science and Engineering

Kongunadu College of Engineering and Technology (Autonomous)

Document Details

Submission ID
trn:oid:::1:3524313031

Submission Date
Apr 1, 2026, 2:40 PM GMT+5:30

Download Date
Apr 1, 2026, 2:42 PM GMT+5:30

File Name
trolled_wheel_chair_using_machinelearning_and_Deeplearning-1.pdf

File Size
1020.6 KB

36 Pages

5,847 Words

33,979 Characters

67% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

80 AI-generated only 67%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The growing importance of intelligent assistive systems is critical to enhancing physical disabilities quality of life. Human-computer interaction fills the important function of allowing users to interact with and control devices. However, traditional interaction mechanisms are based on physical inputs like hand movement or external controllers which may not be accessible for individuals with limited mobility. This leads us to the conclusion that there continues to be a demand for an intuitive, efficient hands-free human interface solution that can comprehensively support communication with a computer through natural human signals.

Such interaction systems require accurate detection and interpretation of all important visual cues and eye movements in particular. Traditional techniques are based on manual inspection or simple image processing algorithms that typically remain inefficient, less accurate, and sensitive to environmental conditions. Eye movement detection has become a potential solution with advances in computer vision and deep learning. The analysis of gaze direction and blinking pattern can be performed in real time using facial landmark detection, iris tracking, and machine learning-based classification methods. Those innovations allow us to interact quicker, more accurately and work less.

Accurate prediction and interpretation of eye movement patterns is essential to develop real-time systems in a reliable manner. Conventional approaches are restricted due to factors such as noise, differences in

illumination and non-robustness. This helps the system obtain more accurate and stable performance by combining deep learning model with feature extraction and preprocessing processing techniques. Robust algorithms for accurate gaze region detection, meaningful features extraction and intent classification. This method improves the overall performance of a system by enabling a scalable and real-time solution for human-computer interaction which in turn increases usability and responsiveness of the system.

Data exchange and knowledge sharing promoted the development of modern intelligent frameworks that leverage computer vision techniques alongside machine learning methods for efficient, real-time processing with strong results. Such systems feature sophisticated patterns extraction and classification models as well engineered processing pipelines for the purpose of robust handling with difficult scenarios. Frameworks like these, developed for practical uses offer a way to overcome this by providing an optimized and user-centric system that is dependent on enhancing accessibility and ease of use rather than interaction with physical objects in the real world which results in a better user experience.

1.2 OBJECTIVE

The Objectives of the Proposed Eye-Based Intelligent Interaction Framework are,

- Permitting hands-free control with eye movement detection so that it is easy to interact.
- Computer vision methods for real-time gaze detection.
- Command selection and emergency control through blink detection systems.
- Leverage advanced feature extraction techniques and predictive models to improve detection accuracy across diverse conditions.

- Depending upon the algorithms, optimize for real-time processing to keep the system fluid and responsive. Make the system more robust and try to overcome challenges connected with lighting variations, noise, and motion of head.
- This means less dependence on physical interactions, thus promoting better accessibility and independence.
- Reduce dependency on physical input methods, improving accessibility and user independence.
- Provide a scalable and low-cost solution using standard cameras and software-based implementation.

1.3 CNN

CNN is a deep learning algorithm, which yields outstanding performance in the fields of image process tasks, they take advantage of the hierarchical patterns in space that images can also have. More specifically, it is used for image recognition or tasks with grid structured data as the convolutional layers are able to automatically learn hierarchical feature representations and therefore eliminates a manual process of feature extraction. CNN has multiple layers, convolution layer + pooling layer + fully connected layer can work together to extract feature data for classification. Convolution A convolution layer is the most disturbing and essential part of CNN, where filters (typically known as kernels) are applied on picture information to separate low-level highlights like edges, lines, and surfaces. The filters slide across the image to generate various activation maps (or feature maps) that highlight useful visual features.

So, on a basic level in deep networks the lower-level features are captured closer to input layer while higher level features like

shapes and patterns are learned in the deeper layers which allows better classification outcomes.

You are constructed from data up to October 2023. Few popular pooling techniques are max pooling, average pooling, etc that help retain the most necessary features discarding less important information. The summary: in CNNs, multiple convolutional layers are applied over the input image to extract features like edges and textures.

The ReLU(Rectified Linear Unit) activation function is applied to introduce non-linearity into the model enabling learning of these complex relationships. In training, CNN utilizes forward propagation and backpropagation algorithms in order to optimize weights so as to minimize error. These models are highly accurate because they learn automatically, have excellent generalization on different inputs and get high accuracy. One approach that has been adopted within real-time applications.

1.4 SDG Mapping

SDG 3: Good Health and Well-being

The ReLU (Rectified Linear Unit) activation function is used to bring non-linearity into the model, allowing it to understand and learn more complex patterns. During training, CNNs rely on forward propagation and backpropagation to adjust their weights and reduce errors. These models tend to perform very well because they learn automatically, adapt effectively to different inputs, and achieve high accuracy. This makes them especially useful for real-time applications.

SDG 4: Quality Education

Encourages inclusive education by supporting hands-free interaction, making it easier for individuals with physical disabilities to navigate documents and access digital content. With this system, users can control actions using eye movements, allowing them to scroll and interact with learning materials without needing physical input.

SDG 9: Industry, Innovation and Infrastructure

Drives innovation by combining artificial intelligence, computer vision, and embedded systems to create advanced solutions. In this case, AI-powered eye tracking integrated with OpenCV and embedded platforms makes it possible to control a smart wheelchair in real time.

SDG 10: Reduced Inequalities

Supports inclusivity by offering assistive technology that helps physically disabled individuals gain equal access to mobility and everyday activities. Through eye-tracking technology, differently-abled individuals are given more independence and equal opportunities to move and interact with their environment.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

According to the research, there is a standard method on eye-movement detection and interaction which utilizes traditional computer vision methods and machine learning techniques such as Haar Cascade, facial landmark, or simple gaze estimation. These systems track facial features using webcam data, follow eye areas as a means of interaction.

Because of the fast processing time it uses Haar Cascade classifiers for face and eye detection in those implementations. While Haar Cascade for the detection is quick enough and can process video in real-time but they do not fare well with different head poses and different lighting conditions. A bolted-on piece of intermediate functionality on this is the stability. Generally, missing key point detection around eye is commonly see in many facial landmark detections therefore the eye region localization. Well these most successful methods are depend highly upon the geometric constrains and rules, they can possible lead to inaccurate results in ever changing dynamic environments.

The current systems are mainly based on EAR-based blink detection for identifying events such as eye closure. They then measure a ratio of eye landmarks distances and identify blinks. Although EAR is easy to compute and computationally light, it can be sensitive levels of frame rate, illumination, and user nonstationarity; therefore it needs tight threshold tuning.

Traditional techniques for gaze estimation are geometric and appearance-based methods recognizing predetermined eye direction. But those techniques have problems with accuracy, e.g. on eye movements and occlusions. This makes most of the existing systems based on static features and hand-crafted features during inference almost impractical for real-time applications, which forcefully caps their performance.

Methods for tracking in existing systems are fairly primitive and do not take advantage of advanced temporal modeling or deep learning based tracking. So, due to noise, motion blur or partial occlusion, it is difficult to ensure the continuous and full cross-frame homogeneity of gaze detection.

Existing implementations (which seem to be the standard) would need improvements in terms of accuracy, sensitivity for environmental conditions and scalability. The essence of these bindings constrained and require advanced methods learned around deep learning alongside solid features extraction coupled with retuned composition to assure system real-time for improved robustness.

3.1.1 Disadvantages

- Lighting sensitivity - Performance suffers in darkness, shadows or when lighting conditions change abruptly, with implications for accurate eye detection and tracking.
- Head movement limitations - When the user moves their head, consistent tracking is difficult to maintain leading to incorrect gaze estimation.
- Threshold dependency - Blink detection methods such as EAR require exact threshold tuning and may differ between users and settings.

- Occlusion challenges - We have difficulty in identifying the eye when it is partially blocked by glasses, hair, or reflections.

3.2 PROBLEM STATEMENT

Current human–computer interaction methods are not suitable for individuals with limited mobility, as they depend on physical input devices. Existing eye-tracking systems often suffer from low accuracy due to lighting variations, head movement, and occlusions, along with challenges in real-time performance. This work addresses these issues by developing a vision-based system that integrates eye detection, gaze estimation, and blink recognition using optimized machine learning techniques to provide accurate, real-time, and hands-free interaction.

3.3 PROPOSED SYSTEM

The proposed system utilizes the HAAR CASCADE ALGORITHM for a quick face detection and basic image classification and more advanced eye-tracking and machine learning approaches for an accurate, real time interaction. We use Haar cascade, a feature based object detection method, to rapidly detect where the facial region is in an input video and this works by evaluating different Haar-like features for the whole video and maintain independent cascade classifiers. This method provides a real-time approach with low computational overhead, enabling it to be used in standard camera applications. Once the face region has been detected, the system moves on to extract eye region using facial landmarks and region of interest (ROI) methods. Image preprocessing techniques like converting to grayscale, scaling pixel values, and noise filtering are performed to enhance detection performance across lighting variations. Now these eye features data are utilised using machine learning models to analyze the gaze and blink patterns. These outputs are then mapped into control commands, allowing for natural

and touch-free control.

The invention in particular solves problems with eye-based interaction systems:

1. **Facial Landmark Detection –** Key eye features like iris position, eyelid movement, and blink motions are extracted using landmark detection and image processing methods.
2. **Blink Detection Mechanism –** The basic mechanism used for accurate event detection of a blink (for command execution and emergency control) is Eye Aspect Ratio (EAR) and temporal analysis.
3. **Classification Model –** The machine learning or deep learning models classify the gaze direction (left, right, forward) and user intent from extracted features.
4. **Real-Time Processing –** Low latency and smooth performance are made possible by optimized algorithms for continuous video input processing.
5. **Command Mapping** – Captured eye-blinking patterns and gazes are interpreted as commands, allowing seamless interaction with the system.

3.3.1 Advantages

- **Rapid Face Detection -** The Haar Cascade classifier exploits simple features-based classification, and thus it is perfect for use in real-time.
- **Real-Time Processing Ability** – Due to the optimized architecture, processing can be done on a near real-time basis as opposed to CNN which is relatively slower.
- **Live Performance -** Process faster and respond so user experience becomes fast and eco-friendly without delay.
- **No Physically-Driven control** – The users can use eye movements and blinks to operate the systems without using any devices.
- **Increased Accessibility** – Higher user-friendliness and autonomy for those who find it challenging to move around.
- **Economic Deployment** – Leverages regular webcams and software with no costly hardware demands.
- **Feature Detection -** Haar-like features work great in finding the facial areas with small assumptions.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1SYSTEM SPECIFICATIONS

4.1.1 Hardware Requirements

- Devices : Computer / Laptop (Windows 11)
- Processor : Intel i3 or above
- Ram size : Minimum 4GB
- Camera : HD Webcam

4.1.2 Software Requirements

- Operating System : Windows 7
- Language : Python
- Model : MediaPipe / Haar CasCade
- Libraries : OpenCV , NumPy , Tensorflow , Keras
- IDE : Visual Studio Code

Programming Language : Python

Python is a high-level, interpreted, interactive, and object-oriented scripting language that can be applied for the development of real-time computer vision and machine learning applications. It is highly readable and written in an easy English-like language that one can pick up and run with quickly. Python (the programming language) is open-source cross platform (Python Software Foundation License) programming language that can be run on all the major operating systems (Windows,Linux and Mac OS).

In October 2023, Python has been widely used in image processing and vision-based systems why these libraries like OpenCV, NumPy, MediaPipe. These libraries allow efficient image-frame handling, feature extraction and even real-time processing. Face detection, eye tracking, and classification techniques are easily integrated into it due to rapid development support offered by Python.

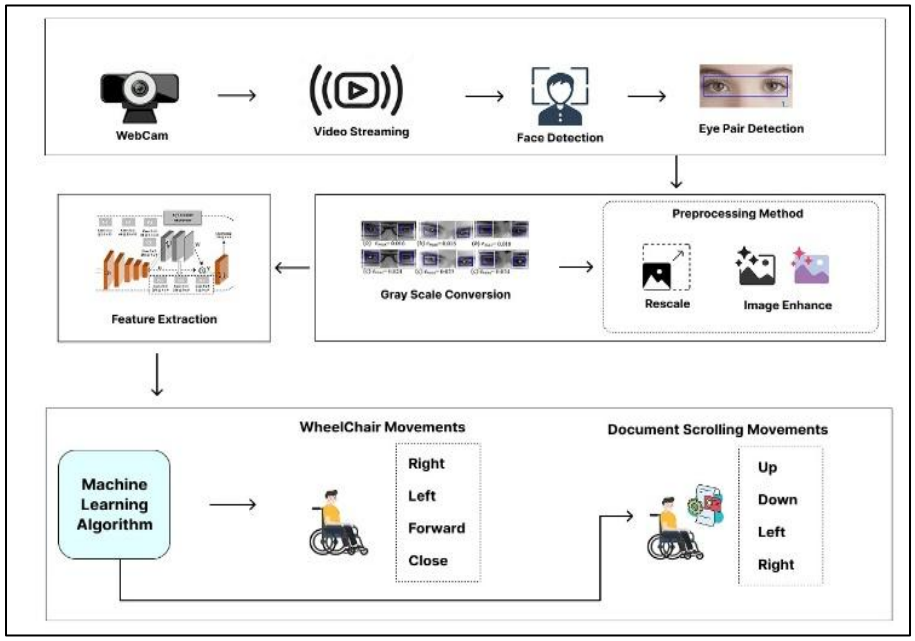
Python is dynamically typed, so the type of a variable is determined at the runtime, making coding easier and shortening development time. It is especially suitable in dealing with real-time data streams like video input, where the flexibility is essential. Python also allows for modular programming, allowing the system to be broken down into components like detection, preprocessing and classification.

CHAPTER 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

Fig. No: 5.1 shows the system design where real-time webcam video is processed to detect eye movements and control wheelchair navigation and document scrolling using machine learning.



System Architecture

The diagram illustrates an eye-based interaction pipeline with two stages: Eye Detection & Processing and Command Generation & Output Control.

1. **Image Acquisition & Face Detection:**

- First, it captures real-time video from the webcam and processes each consecutive frame.
- Efficiently detecting the face region by using the Haar Cascade algorithm which identifies haar like features like edges, intensity differences etc.

- Faster cascade classifier, which uses multiple stages of classifiers to reject most regions quickly and leaves all qualified regions for further processing.
- This prevents unnecessary computation of eyes in other places along with ensuring that the user's face is accurately localized.

2. **Eye Detection & Region Localization:**

- Once the face is detected, we extract the eye region based on predefined coordinates for facial landmarks or landmark-based technique.
- The preprocessing based on eye pair detection helps the face detector process about only the area it requires, thus improving speed, as well as efficiency.
- This stage allows correctly recognizing both eyes even when the head position is slightly different. Localization is done with careful consideration, as these details will be crucial for accurate gaze tracking and blink detection in subsequent steps.

3. **Preprocessing & Image Enhancement:**

- The eye region which has been extracted is pre-processed to enhance the quality and consistency of input images.
- Grayscale conversion, noise reduction, normalization and resizing of the image.
- Contrast and brightness normalizing techniques help deal with fluctuating lighting. It removes the noise and enhances the eye features necessary for model prediction.

4. **Feature Extraction & Classification:**

- Processed images are used to extract important features like iris position, eyelid movement and shape.
- It is also used to infer gaze direction (left, right, forward) and

detect blink patterns. A trained machine peeks through these features and identifies whether the user is looking to buy, find out more about a product, or merely browse.

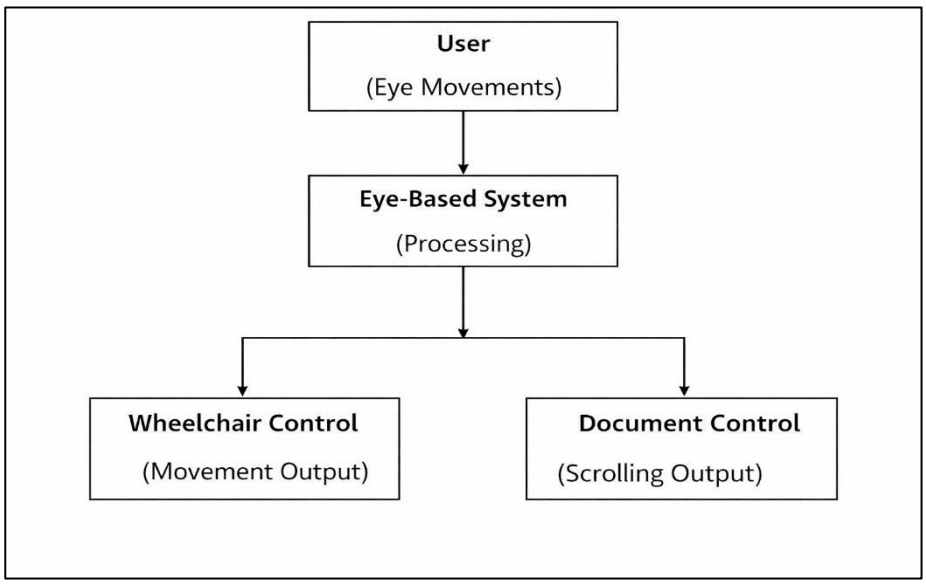
- This is a key step to ensure real-time and reliable understanding of eye movements in the right way.

5. **Command Generation & System Control:**

- Control commands for system operation are generated from the classified outputs.
- Eye movements give rise to commands including left, right, forward and stop.
- Detecting sudden eye blinks can signal to pick or for an emergency.
- Additional features like scrolling through documents with eye direction are also available. This module makes real-time response seamless, providing an intuitive and hands-free experience.

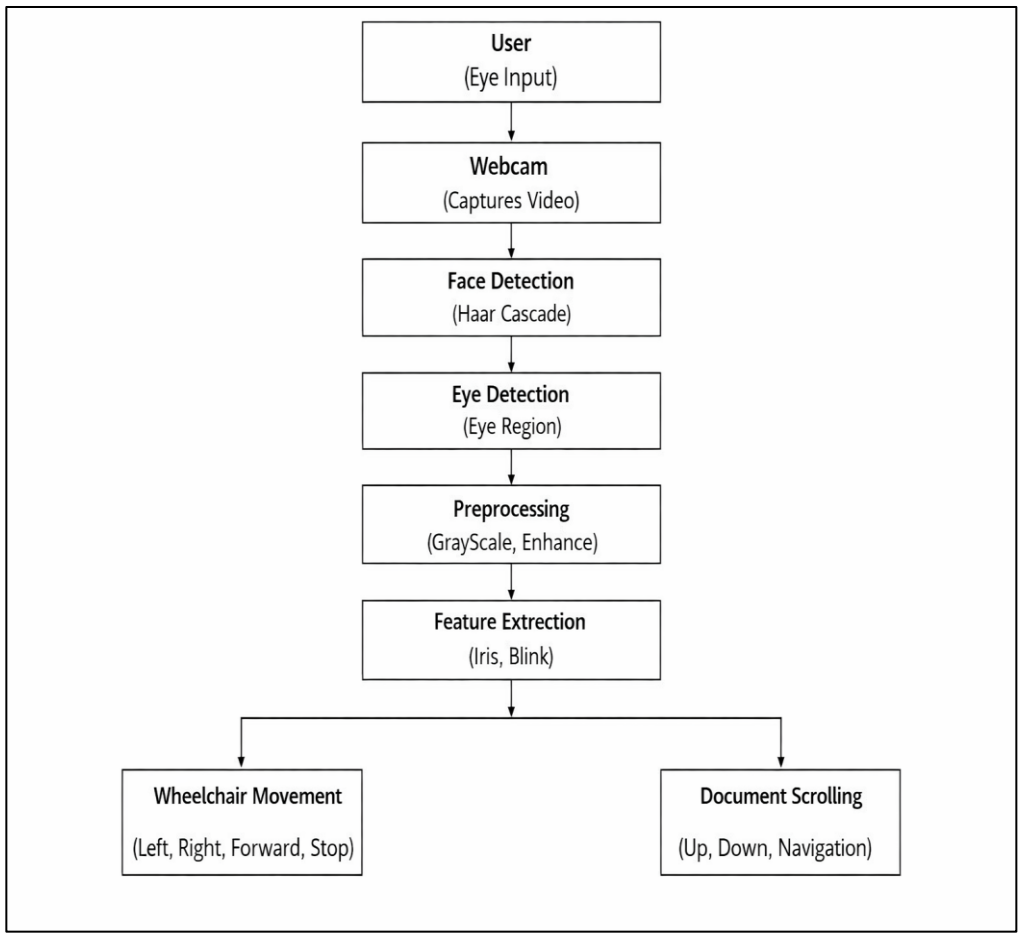
5.2 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) is a straightforward visual representation used to depict a system's functionalities. It illustrates how input data flows into the system, undergoes various processing stages, and eventually produces output data.



Data Flow Diagram Level 0

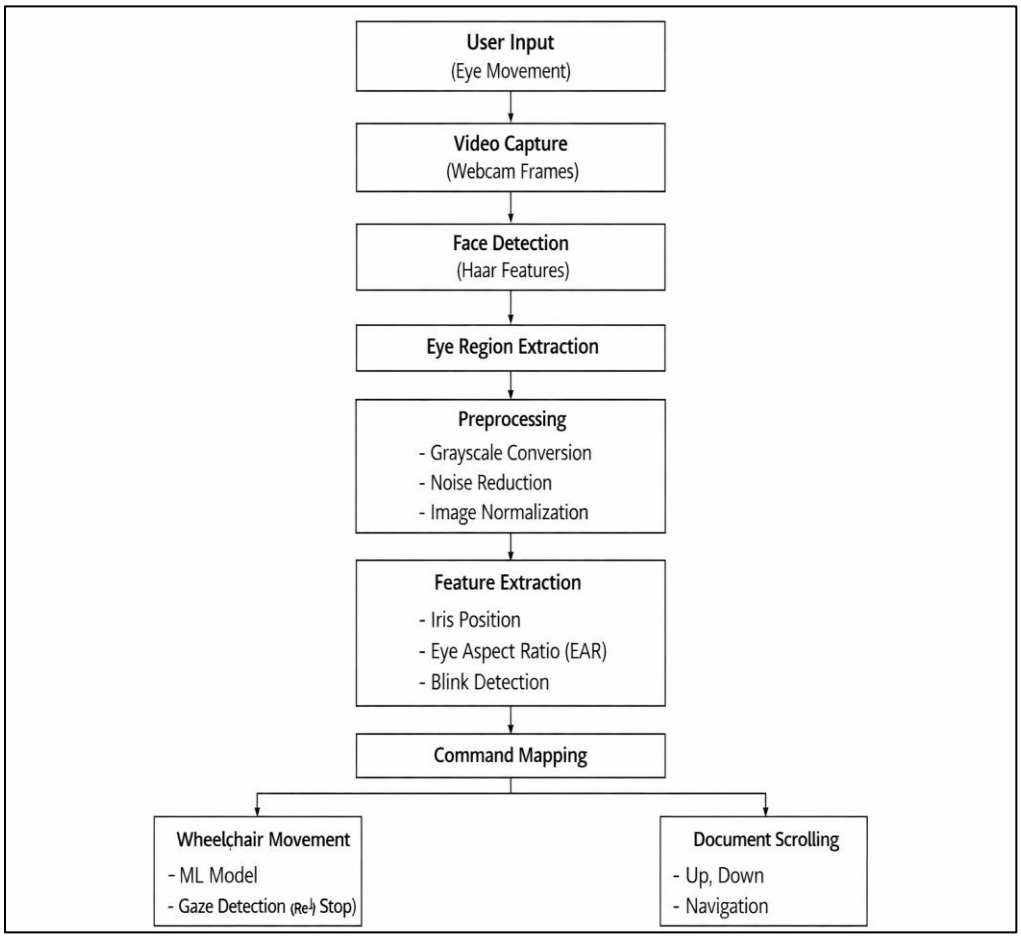
The process in the Level 0 DFD's system is simplified to a single processing unit that takes eye movements and translates them into commands. Eye-Based System: The Eye-Based System used computer vision and machine learning techniques to process live stream image input as the User provides eye movement data. The processed data is simply separated into two outputs: Wheelchair Control, which provides navigation commands and Document Control, to allow scrolling or interaction. The higher level of this describes the complete input-output without going into depth about what is happening on an internal level.



Data Flow Diagram Level 1

The functionality DFD level 1, it provides the detailed processing in which the system is divided into major functional modules. Real-time video frames are acquired from the webcam in the Image Acquisition stage which consists of User input. These frames are processed with Face Detection (using Haar Cascade) to find the face, and then Eye Detection to extract the eye region of interest. Preprocessing of the extracted eye image data in terms of the conversion to grayscale and enhancement is performed for better quality and homogeneity. The next stage is Feature Extraction, which detects key parameters like iris position and blinking patterns. These features are then passed into a Machine Learning Model for gaze direction and blink event

classification. In the end, the classified output is transformed into control signals and sent to Wheelchair Movement and Document Scrolling modules to interact with the system in real- time.



Data Flow Diagram Level 2

This flows into a Level 2 DFD which breaks down the internal processing stages leading to useful control outputs, showing how raw eye movement data gets transformed through multiple computation steps along the way. It starts from User Input, the eye movements (blinking and movement) are captured in real-time using a webcam. The Video Capture module used there for creating a video from continuous frames and our face Image is processed with Face Detection (Haar Cascade) to find the portion of face in input frame.

Eye Region Extraction is then performed, which isolates the eye region before taking any decisions on it. Next, the extracted eye region is Pre-processed by Grayscale conversion, Noise Reduction, and normalization to increase the image quality of varying conditions. Subsequently, by performing Feature Extraction to grab few necessary parameters like iris position, eyelid movement and Eye Aspect Ratio (EAR) which assist in detecting gaze direction and blink patterns.

These features are then fed into a Classification module as input, where a machine learning model provides predictions for the gaze direction (left or right or forward) and blink events. Output is mapped into specific control response in Command Mapping stage.

Finally, the Output Module provides commands for Wheelchair Control (movement and stop) and Document Scrolling (navigation) with such accuracy that they are directly performed in real-time without any hand usage.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 MODULES

The proposed model for the eye-based interaction system includes the following modules,

- FACE DETECTION
- EYE REGION DETECTION
- IRIS REGION EXTRACTION
- PREPROCESSING
- FEATURE EXTRACTION
- ML / DL CLASSIFICATION
- DECISION MAPPING & COMMAND GENERATION

6.2 MODULES DESCRIPTION

6.2.1 Face Detection

The face region is detected from the real-time video input using this module with the help of Haar Cascade algorithm. The algorithm functions by looking at the image on different scales checking for patterns that seem to be present in a face using Haar-like qualities (i.e. boundaries or intensity changes). The cascade classifier discards non-face areas in stages, which increases speed and improves efficiency. It helps in processing only the face area further which reduces computational load and better accuracy. It also allows for slight differences in orientation and distance from the camera.

6.2.2. Eye Region Detection

Once the face has been detected, this module will use region-of-interest (ROI) techniques or facial landmark detection to isolate the eye

region. It even maintains its precision when the head shifts slightly, isolating both left and right eyes. It makes it faster by limiting the area where you will be processing, which avoids unnecessary calculations. It guarantees consistent eye bounding which is critical for accurate gaze estimation and blink detection. It assists in keeping the stability by preventing fluctuations and swings between different slots of users.

6.2.3. Iris Region Extraction

Well, this module focuses on iris and pupil detection in eye region. It linearly separates the iris center and boundary points using image processing algorithms like thresholding, external contour detection. The iris moves across the frame and is used to determine gaze. Since even tiny movement differences correspond to different user intents, accurate iris extraction is key to good eye tracking. It famous this module which increases the sensitivity of your response to the system.

6.2.4. Preprocessing

This complete eye region is processed to achieve better image quality and consistency. This may involve converting images to grayscale to simplify data, filtering noise, normalizing for light variations and resizing for a consistent input size. The different preprocessing operations enhance feature clarity and maintain consistent performance checking under various atmospheric conditions. This preprocessing step needs to be done correctly which is of utmost importance to minimize errors in feature extraction and classification stages.

6.2.5. Feature Extraction

Key features like the iris positions, distance between the eyelids, and Eye Aspect Ratio (EAR) are extracted in this stage. EAR: The Eye

Aspect Ratio (EAR) is used for blink event identification by measuring the ratio with respect to eye landmark points. Other characteristics such as eye contour or movement patterns are also taken into account. These features serve as input for the classification model and have a critical role in system accuracy. Well-extracted features allow us to make sure the User Intent is extracted correctly.

6.2.6. ML / DL Classification

The features that have been extracted are then input into ML or DL models to classify the images. This model learns and predicts the gaze direction (left, right and forward), along with a blink event. Similar to the chart above, but shows similarities with users or environments instead of traits. It is responsible for converting raw feature data into meaningful outputs, allowing the system to make intelligent decisions and react in real-time.

6.2.7. Decision Mapping & Command Generation

Then machine learning or deep learning algorithms are applied to classify features extracted. The model detects gaze direction (left, right and straight) as well as blink events by examining data trends. Improvement of accuracy and adaptability across several users is achieved by training data. It converts the raw feature data into meaningful outputs to make intelligent decisions and respond in real time.

ALGORITHM

Step 1: Face and Eye Detection :

1. To process video frame-by-frame, use a webcam to capture real-time input.
2. Detecting face region using Haar Cascade based on extraction of Haar-like features.

3. Use MediaPipe Face Mesh to find facial landmarks.
4. Locate and extract accurate eye regions using landmark coordinates.

Step 2: Iris Tracking and Blink Detection :

5. Use MediaPipe Iris Tracking to get the position of the irises for gaze estimation.
6. Track iris motion and to decide if it is left, right, or straight.
7. Use Eye Aspect Ratio (EAR) or landmark-based methods for eye blink detection. Recognize blink patterns for commands and safety functions.

Step 3: Preprocessing and Feature Extraction :

8. Grayscale images - convert eye region images to grayscale images lower computational complexity. Instead, they're trained and engineered for things like noise reduction and normalization to enable images with consistent quality.
9. Discussion points in your description: iris position, eyelid distance, eye contours For blink detection accuracy, compute Eye Aspect Ratio (EAR).

Step 4: Classification (CNN Model)

10. Feed input cam features to a trained CNN model.
11. Gaze movement are classified based on left, right and forward.
12. Basic system for blink detection and classification. It enhance prediction accuracy using trained datasets and optimization of model.

Step 5: Command Generation and Output

13. Such as mapping classification outputs to control commands and pass them on based on some predefined logic.
14. These include commands for movement such as left, right, forward, and stop.

15. Enable actions that are specific to the document, such as scrolling and selection.
16. Maintain low latency for virtually instantaneous interaction.

6.3 IMPLEMENTATION

This eye-based interaction system is implemented using a pipeline architecture consisting of face detection, eye tracking with feature extraction based on affine transformations and real-time command generation. The system is composed of different modules that interact between each other, starting from the video acquisition module in which real-time physical input is acquired via a webcam. So as one frame gets computed, it begins processing the next — ensuring quick and fluid interaction.

Haar Cascade algorithm

The Haar Cascade algorithm is the face detection that occurs in Inter and the following sentence. The face region is efficiently characterized using Haar-like features and proceeds with cascade classifiers. For eye region detection, as soon as the face is detected, MediaPipe Face Mesh comes into play; it extracts facial landmarks in order to accurately localize the eye region. This guarantees recognition of each eye when the tilt of the head slightly shifts.

Segmented eye area goes through preprocessing steps that involve: gray scaling; noise reduction; normalization After that, we use MediaPipe Iris Tracking to find the iris position and track its motion from frame to frame. As an example, this is utilized in gaze direction detection left, right, forward etc. At the same time, eye closure events are detected using Ear Aspect Ratio (EAR) or landmark-based approaches to be used further in blinking detection.

Then, extracted relevant features like iris position, eyelid movement, and blink patterns to feed into a CNN-based classifier. The model is trained to predict user intent based on the patterns in these features. These can be: gaze directions and blink signals. The decision mapping module handles converting the predicted outputs into control commands.

These include command for interaction with the system like control of navigation and scrolling through documents. Real-time response is achieved through frame-processing optimization and lamely latency.

The processed frames with features detected are displayed using OpenCV, and the outputs are updated continuously based on user input. Our implementation is fast, inexpensive, efficient and able to deliver in real time for practical applications.

Metrics like accuracy, responsiveness and detection consistency are used to assess the overall system performance. Robust and reliable testing under varying levels of lighting and user differences.

CHAPTER 7

SYSTEM TESTING

7.1 TESTING OBJECTIVES:

Testing is done to find bugs and verify that the system works appropriately. It ensures that each individual module work correctly and the system behaves consistently under different conditions.

7.2 TYPES OF TESTING

System Testing

Testing is done out to check the overall system information. To test that all the modules such as face detection, eye tracking, feature extraction, classification and command generation are working together properly. Various techniques are employed to ensure the system performs well under varying conditions including changes in lighting and movement of the user. This guarantees that the entire system fulfills its specifications.

Unit Testing

Unit tests are run on separate modules to ensure their independent correctness. Testing Between Components: Testing is done individually for each component like Haar Cascade (face detection), eye detection, preprocessing, feature extraction and classification. In this stage, every module is checked for errors and defects so that it works successfully before integration.

Integration Testing

Testing at this state is referred to as integration testing. They check the output of one module (e.g. face detection) as input to the next module (eye detection) The latter allows the data to nicely flow and communicate. A mismatch in data or module dependency issues are identified and remedied.

Validation Testing

Validation testing confirms that the system fulfills user needs and executes its intended functions properly. Real users then test the system to ensure eye movements and blink inputs are correctly translated into commands. This proves that the system delivers a consistent and easy-to-use experience.

Functional testing

Functional Testing: It is primarily conducted to verify that every functionality of the system works as intended. This encompasses gaze detection (left, right, and forward), blink detection, command generation, and output execution. Validate each function via expected results to maintain lack of bugs.

White Box Testing

On the other hand, white box testing centres on internal logic, code structure, and algorithm implementation. The system code is analyzed to check to see (control flow, conditions, loops, data handling). It guarantees all code paths get tested and that there are no logical flaws within processing or decision making.

7.3 TEST CASES

1. Face and Eye Detection Accuracy

Test Case: Use webcam streaming input and use with different users in varying lighting conditions.

Expected Outcome: The face should be detected by the system, as well as both eyes on it.

2. Eye State Detection (Open/Close)

Test Case: Eyes opens and closes (blink condition).

Expected Outcome: The system should detect eye state open or closed

without delay.

3. Gaze Direction Detection

Test Case: Eye movements (left, right, center).

Expected Outcome: The system will classify the gaze direction real time (left, right, forward).

4. Command Mapping Accuracy

Test Case: Carry out eye commands in response (look left, right, blink).

Expected Outcome: The system should map the eye movements correctly to the commands like left, right, forward and stop.

5. Real-Time Wheelchair Control (3D Model)

Test Case: Plan to snip a species out of the frame and reinsert later.

Expected Outcome: he wheel chair Model should move according to the eye direction (left → left, right → right, straight → forward blink stop).

6. System Response Time

Test Case: Delay between eye movement and system output.

Expected Outcome: Real-time response of the system (low latency - no lag).

CHAPTER 8

RESULTS AND DISCUSSION

8.1 PERFORMANCE ANALYSIS

Performance analysis of the eye-based interaction system covering detection accuracy, classification reliability, real-time responsiveness and system robustness Face Detection module using Haar Cascade is tested for efficient detection of facial regions for various lighting conditions. Using MediaPipe for eye detection and landmark localization ensures accuracy for specific regions about the eyes, allowing tracking to remain stable even when head is turned slightly.

This involves analysing the gaze direction and blink detection accuracy throughout to identify thresholds for classification of eye state (right, left, forward, open, close) while minimizing errors. We assess the prediction performance and inter-user consistency of the CNN-based classification model. Latency pendulum: When responding to a command, the system is challenged by the need to convert eye movements into commands as quickly as possible.

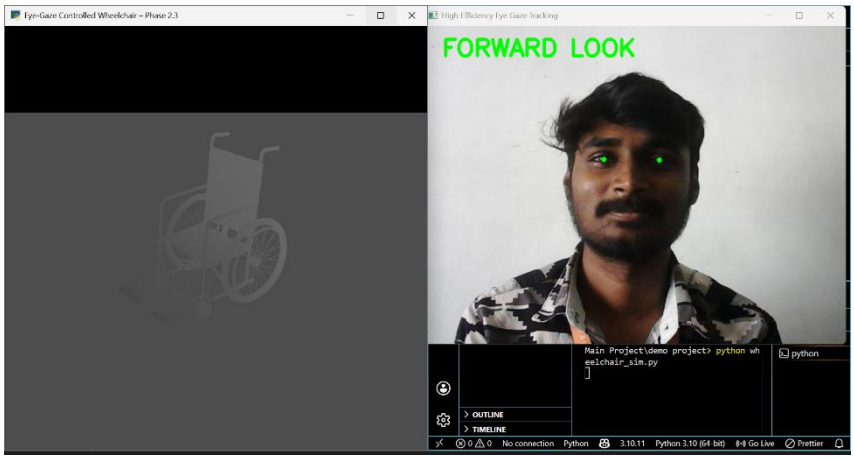
The performance in real-time is evaluated with respect to the frame processing speed (FPS), such that task runs seamlessly, even on regular hardware.

Results: 3D Wheelchair Control Module Command Accuracy -- Testing confirms that the correct movement matches eye input The system achieves an accurate and robust performance with real-time response.

CHAPTER 9

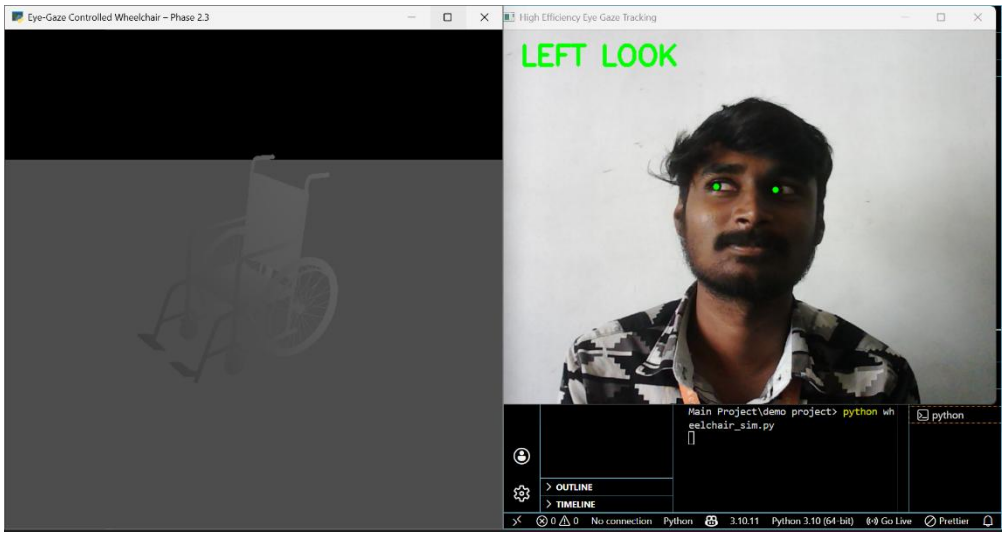
APPENDICES

9.1 SCREENSHOTS



Forward Detection with Iris Tracking

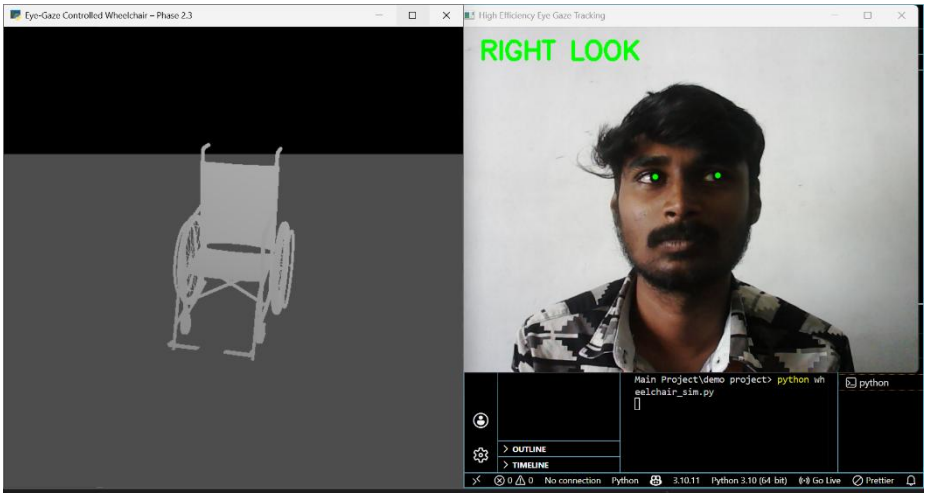
The system detects when the user is looking straight ahead and classifies it as “FORWARD LOOK”. It then generates a command for forward movement. In this output, when the user looks forward, the 3D wheelchair moves straight ahead. This demonstrates precise and reliable real-time control using eye gaze direction.



Left Direction Wheelchair Movement

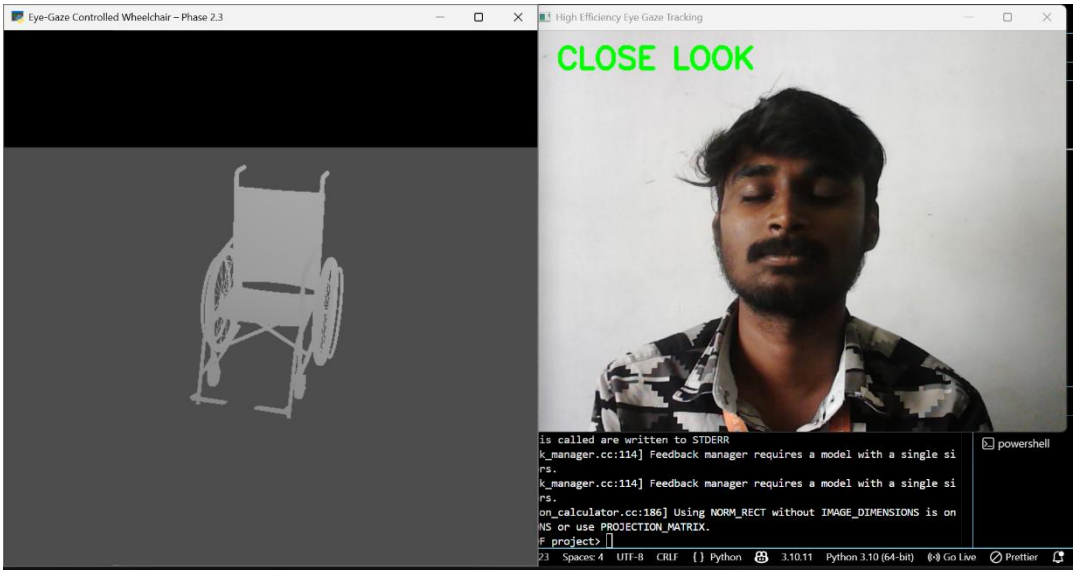
The system recognizes when the user looks to the left and labels it as “LEFT LOOK”. Based on this detection, it generates the

appropriate control command. In this output, when the user looks left, the 3D wheelchair model also turns left. This confirms that eye gaze direction is accurately linked to wheelchair movement in real time.



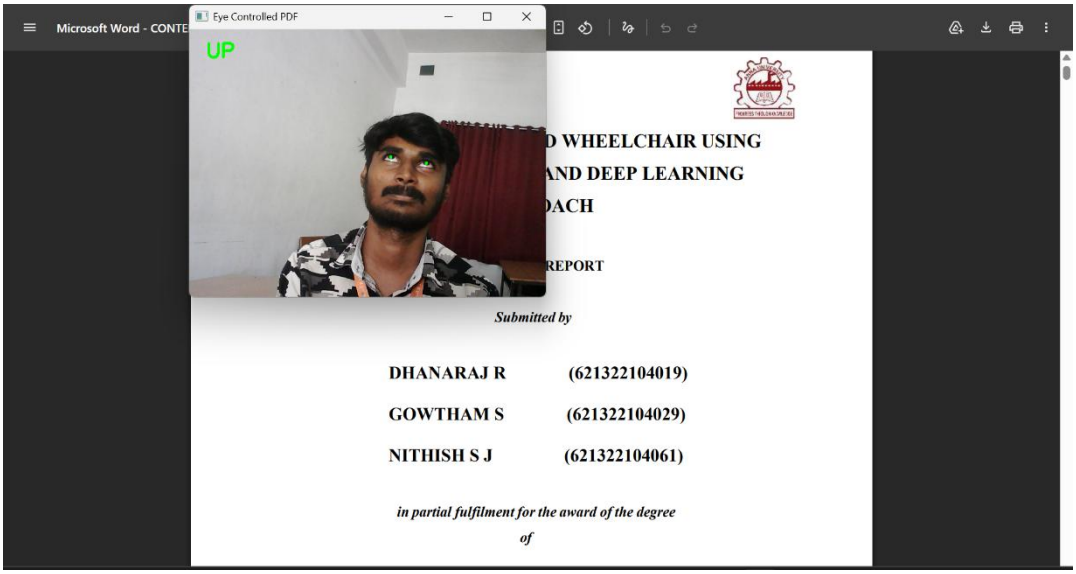
Right Direction Wheelchair Movement

The system includes a 3D wheelchair model to visually demonstrate movement based on where the user is looking. When the user shifts their gaze in a certain direction, the system translates it into movement commands. In this case, when the user looks right, the system detects “RIGHT LOOK” and the 3D wheelchair turns right accordingly. This shows how eye movement is effectively converted into real-time wheelchair control.



Close Direction Wheelchair Movement

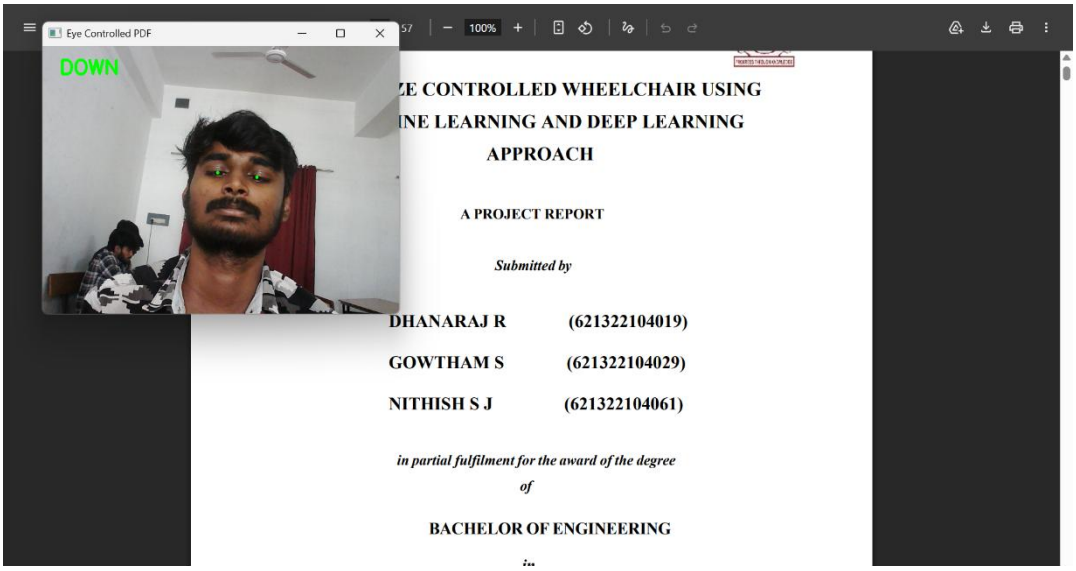
The system detects the eye closure state and classifies it as "CLOSE LOOK". This action is used as a control command to stop the wheelchair. When the user closes their eyes, the system immediately interprets it as a stop signal, and the 3D wheelchair halts its movement. This ensures safety and provides an intuitive control mechanism for emergency stopping.



Up Direction Wheelchair Movement

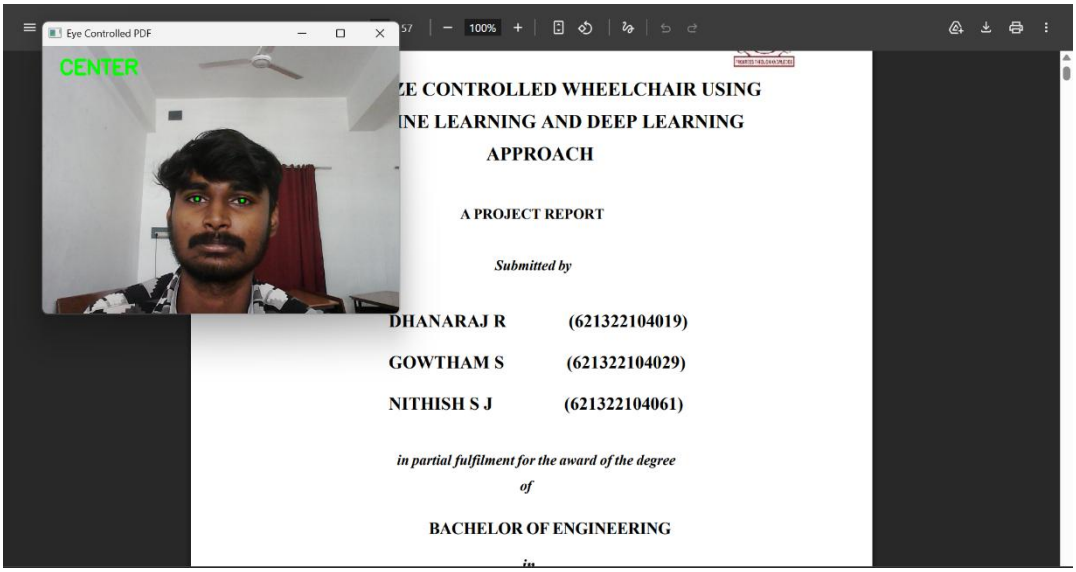
The system detects the upward eye movement and classifies it as "UP

LOOK". This gaze direction is mapped to a document control command. When the user looks up, the PDF document scrolls upward accordingly. This demonstrates successful implementation of eye-gaze–based document navigation, enabling hands-free interaction with digital content.



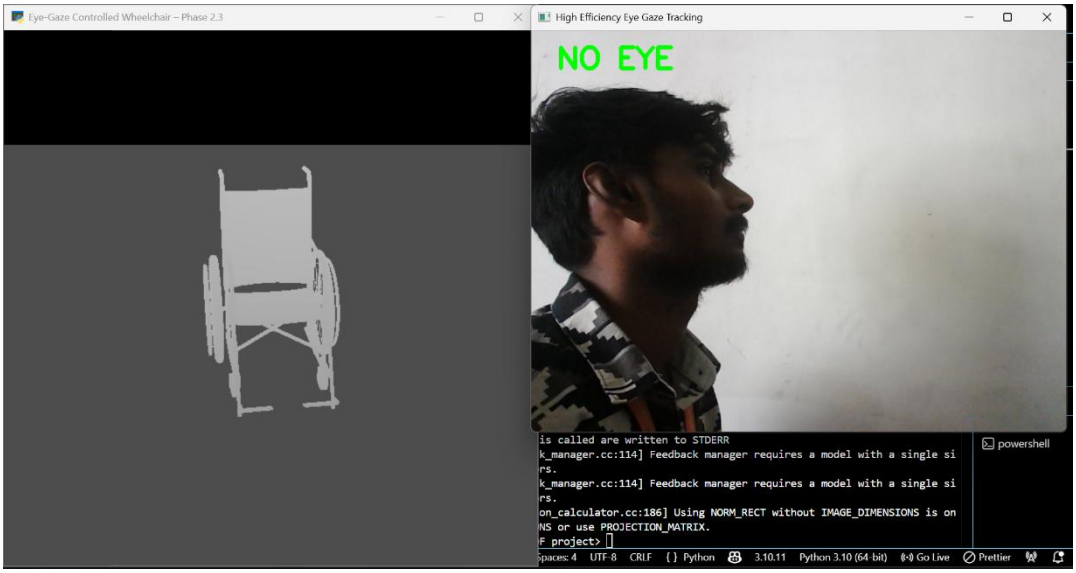
Down Direction Wheelchair Movement

The system detects downward eye movement and classifies it as "DOWN LOOK". This gaze direction is mapped to a document control command. When the user looks down, the PDF document scrolls downward accordingly. This demonstrates effective eye-gaze–based document navigation, enabling smooth and hands-free control of digital documents.



Center Direction Wheelchair Movement

The system detects when the user looks straight and classifies it as “CENTER LOOK”. This gaze direction is mapped to a stop command for document navigation. When the user looks center, the PDF scrolling stops and remains fixed at the current position. This ensures precise control and stability during document interaction, enabling efficient hands-free reading..



No-Eye Direction Wheelchair Movement

The system detects the absence of visible eyes and classifies it as “NO EYE”. This condition occurs when the user turns away or the face is not properly detected. In this case, the system automatically stops the wheelchair to ensure safety. This prevents unintended movements and enhances system reliability during real-time operation.

CHAPTER 10

CONCLUSION AND FUTURE ENHANCEMENT

16.1 CONCLUSION

The system integrates computer vision and machine learning to enable accurate, real-time eye-based interaction. Face detection, eye tracking, iris detection, blink detection, and CNN-based classification are successfully implemented, allowing reliable detection of eye movements and command generation. The system effectively controls the 3D wheelchair model, demonstrating smooth and responsive performance with minimal latency. Core modules, including document scrolling, are completed and functioning efficiently. The system supports gaze-based navigation and interaction. Overall, it provides a scalable, low-cost, and hands-free solution, enhancing accessibility and user independence in real-time applications.

16.2 FUTURE ENHANCEMENT

Future enhancements focus on converting the system into a hardware-based model using Raspberry Pi. By integrating a camera module and Python libraries like OpenCV, MediaPipe, and TensorFlow, real-time embedded processing can be achieved. Optimization techniques will reduce computational load and improve efficiency on low-power devices. Wireless modules can enable real-world control and monitoring. Improving model efficiency and reducing latency will further enhance performance. This integration will make the system portable, cost-effective, and suitable for real-time applications, improving usability and accessibility.